Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la
Lucrarea de laborator Nr. 6
Disciplina: Tehnici de programare
Tema: ”Tutorial LearnPython.org”

A efectuat: st. gr. SI-264 Gaju Daniel
A verificat asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

> . **Construim** două unelte de forensics, scrise de voi în Python, care intră în arsenal. Lucrarea A este un decoder universal: dat un șir, recunoaște stratul (base64, hex, URL), îl desface, iar dacă e un XOR cu cheie scurtă sau un hash de parolă, îl sparge prin forță brută sau dicționar. Lucrarea B este forensics pe fișiere: citești semnătura reală, scoți textul și metadatele (inclusiv GPS) dintr-o poză, extragi un fișier ascuns, apoi ascunzi și dezvălui un mesaj prin steganografie LSB. Amândouă se predau cum cere regula de fier: cod și output, niciodată o captură de ecran.

Lucrarea A: Decoderul Universal

A1. Text și octeți: încălzirea

Am diferențiat textul citibil (string) de reprezentarea sa brută în memorie (bytes). Criptografia și decodările lucrează la nivel de octeți, unde fiecare element este un număr între 0 și 255.

A2. Recunoşti stratul după formă.

Am folosit expresii regulate (re) pentru a detecta tiparul caracterelor dintr-un șir și a ghici codarea. Ordinea este importantă: verificăm întâi URL (are %), apoi HEX (doar caractere 0-9 și a-f, lungime pară) și la final Base64.

```
4   def ghici_strat(s):
5       if "%" in s and re.search(r'%[0-9a-fA-F]{2}', s): return "url"
6       if re.fullmatch(r'[0-9a-fA-F]+', s) and len(s) % 2 == 0: return "hex"
7       if re.fullmatch(r'[A-Za-z0-9+/=]+', s): return "base64"
8       return "necunoscut"
```

Fig. 1.2

A3. Desfaci stratul recunoscut

Pe baza tipului detectat la A2, am aplicat funcția specifică de decodare din bibliotecile standard Python (base64.b64decode,bytes.fromhex, unquote).

```
10  def desfa(s):
11      strat = ghici_strat(s)
12      try:
13          if strat == "base64": return strat, base64.b64decode(s)
14          if strat == "hex": return strat, bytes.fromhex(s)
15          if strat == "url": return strat, unquote(s).encode()
16      except: pass
17      return strat, s.encode()
```

Fig. 1.3

A4. Straturi multiple, într-o buclă

Deoarece mesajul inițial a fost împachetat de mai multe ori (ex: text -> hex -> base64), am pus decodarea într-o buclă. Ne oprim când rezultatul conține doar caractere ASCII printabile (verificate prin funcția citibil).



```
for _ in range(8):
    strat, b = desfa(val)
    if citibil(b):
        rez = {"strat": strat, "rezultat": b.decode()}
        with open("arsenal/forensics/fisa.json", "w") as f:
            json.dump(rez, f)
        print(f"Text decodat salvat în fisa.json: {rez}")
        break
    val = b.decode(errors="ignore")
```

Fig. 1.4

A5. Spargi un XOR cu cheie de un octet

O cheie de un octet are doar 256 de valori posibile (0-255). Am iterat prin toate, am aplicat operația logică XOR (^) pe fiecare octet din fișier și am verificat dacă rezultatul obținut arată a text citibil.



```
try:
    date = open("probe/xor.bin", "rb").read()
    for k in range(256):
        clar = bytes(b ^ k for b in date)
        if citibil(clar):
            print(f"XOR cheie gasita: {k} -> {clar.decode()}")
except FileNotFoundError: pass
```

Fig. 1.5

A6. Spargi un hash prin dicționar

Am dedus algoritmul de hashing pe baza lungimii șirului (32 caractere pentru MD5, 40 pentru SHA1, 64 pentru SHA256). Am criptat pe rând fiecare cuvânt dintr-o listă (wordlist) și am comparat rezultatul cu hash-ul țintă.



```
    try:
        tinta = open("probe/hashuri.txt").readline().strip()
        alg = "md5" if len(tinta) == 32 else "sha1" if len(tinta) == 40 else "sha256"
        for cuv in open("probe/wordlist.txt", encoding="latin1"):
            cuv = cuv.strip()
            h = hashlib.new(alg)
            h.update(cuv.encode())
            if h.hexdigest() == tinta:
                print(f"Parola gasita ({alg}): {cuv}")
                break
```

Fig. 1.6

A7. argparse, fișa JSON și un test

Am adăugat suport pentru argumente în linia de comandă, am salvat rezultatul decodării într-un format structurat (.json) și am scris un test unitar automat cu assert.



```

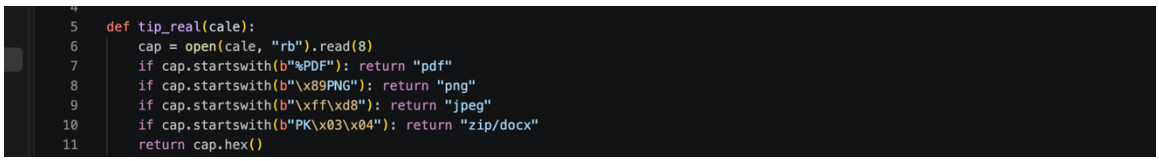
def test_base64():
    strat, b = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert b == b"Salut"
```

Fig. 1.7

Lucrarea B: Forensics și Steganografie

B1. Magic bytes (Tipul real al fișierului)

Extensiile pot fi falsificate. Am deschis fișierele în mod binar (rb) și am citit primii octeți (semnătura) pentru a stabili formatul real al fișierului (ex: un fișier ZIP/DOCX începe mereu cu PK).



```

def tip_real(cale):
    cap = open(cale, "rb").read(8)
    if cap.startswith(b"%PDF"): return "pdf"
    if cap.startswith(b"\x89PNG"): return "png"
    if cap.startswith(b"\xff\xd8"): return "jpeg"
    if cap.startswith(b"PK\x03\x04"): return "zip/docx"
    return cap.hex()
```

Fig. 1.8

B2. Extragerea textului citibil (strings)

Am folosit regex direct pe datele binare brute pentru a găsi secvențe de minimum 4 caractere imprimabile consecutive,utile pentru a extrage parole ascunse sau indicii.

```
15  date_png = open("probe/ascuns.png", "rb").read()
16  for m in re.findall(rb"[ -~]{4,}", date_png):
17      pass
```

Fig. 1.9

B3. Metadate EXIF și GPS din poză

Am extras datele ascunse adăugate automat de aparatul foto sau telefon. Am căutat specific blocul cu eticheta "GPSInfo" (tag-ul 34853) care conține coordonatele unde a fost făcută fotografia.

```
18  exif = Image.open("probe/foto.jpg")._getexif() or {}
19  for id_tag, val in exif.items():
20      if TAGS.get(id_tag, id_tag) == "GPSInfo":
21          print("Coordonate GPS (tag 34853):", val)
```

Fig. 1.10

B4. File carving (Extragerea fișierului ascuns)

Am căutat semnătura magică a unei arhive ZIP (`PK\x03\x04`) în interiorul structurii binare a unei imagini PNG. După ce i-am găsit poziția (indexul), am tăiat restul octeților și i-am salvat într-un fișier separat valid.

```
23  i = date_png.find(b"PK\x03\x04")
24  if i != -1:
25      open("gasit.zip", "wb").write(date_png[i:])
26      print(f"Arhiva extrasa salvata la 'gasit.zip', decupata de la octetul {i}")
```

Fig. 1.11

B5. Ascunderea unui mesaj prin LSB

Am injectat un mesaj secret modificând doar ultimul bit (cel mai puțin semnificativ) al fiecărui canal de culoare (RGB) din pixeli. Vizual, imaginea rămâne neschimbată. Am forțat ultimul bit la 0 prin mascare (& ~1), apoi l-am înlocuit cu bitul mesajului nostru.

```
7   biti_mesaj = "".join(f"{x:08b}" for x in mesaj)
```

Fig. 1.12:

B6. Dezvăluirea mesajului LSB

Am citit imaginea modificată și am extras ultimul bit din fiecare culoare (r & 1). Am regrupat acești biți câte 8 pentru a forma octeți, oprindu-ne când am întâlnit octetul de valoare 0 (adăugat intenționat la B5 ca delimitator de final).

```
20  px = list(Image.open("stego.png").convert("RGB").getdata())
21  biti_extrasi = ""
22  for r, g, b in px:
23      biti_extrasi += str(r & 1) + str(g & 1) + str(b & 1)
24
25  octeti = [int(biti_extrasi[i:i+8], 2) for i in range(0, len(biti_extrasi), 8)]
26  mesaj_decodat = ""
27  for o in octeti:
28      if o == 0: break
29      mesaj_decodat += chr(o)
30
31  print("Mesaj LSB dezvaluit:", mesaj_decodat)
```

Fig. 1.13

CONCLUZII

În concluzie, această lucrare de laborator mi-a consolidat înțelegerea modului în care informația este reprezentată, ascunsă și recuperată la nivel de octeți, oferindu-mi instrumente practice pentru investigațiile de tip forensics.

Prin realizarea primei părți (Decoderul Universal), am învățat să manipulez date binare, implementând un sistem capabil să recunoască și să decodifice automat multiple straturi de codare (Base64, Hex, URL). De asemenea, am aplicat concepte fundamentale de criptanaliză, reușind să sparg un mesaj criptat cu XOR prin metoda forței brute (pe un spațiu de 256 de chei posibile) și să recuperez o parolă dintr-un hash prin intermediul unui atac cu dicționar.

În a doua parte (Forensics și Steganografie), am explorat tehnici esențiale de analiză a fișierelor. Am realizat că extensiile pot fi înșelătoare și am învățat să determin tipul real al fișierelor pe baza semnăturilor (magic bytes). Extragerea metadatelor EXIF a demonstrat riscul de privacy asociat partajării imaginilor cu coordonate GPS, în timp ce aplicarea tehnicii de file carving a dovedit că fișiere întregi pot fi atașate în structura aparent normală a altora. Nu în ultimul rând,implementarea steganografiei LSB mi-a arătat practic cum datele pot fi disimulate și ulterior extrase direct din biții de culoare ai pixelilor, fără a altera aspectul vizual al imaginii.